



EMBEDDED AI · PROJECT REPORT

Real-Time Vehicle Detection & Traffic-Congestion Monitoring on the Raspberry Pi 5

A YOLOv8n detector compressed for the CPU-only edge with structured pruning and INT8 / FP16 quantization, for a Safe City adaptive traffic-signal application.

Field	Detail
Student	Engr Danial Azam
Roll No	PHDEE25003
Instructor	Prof Rehan Hafiz
Course	Embedded AI
Target Device	Raspberry Pi 5
Base Model	YOLOv8n (Ultralytics)

Contents

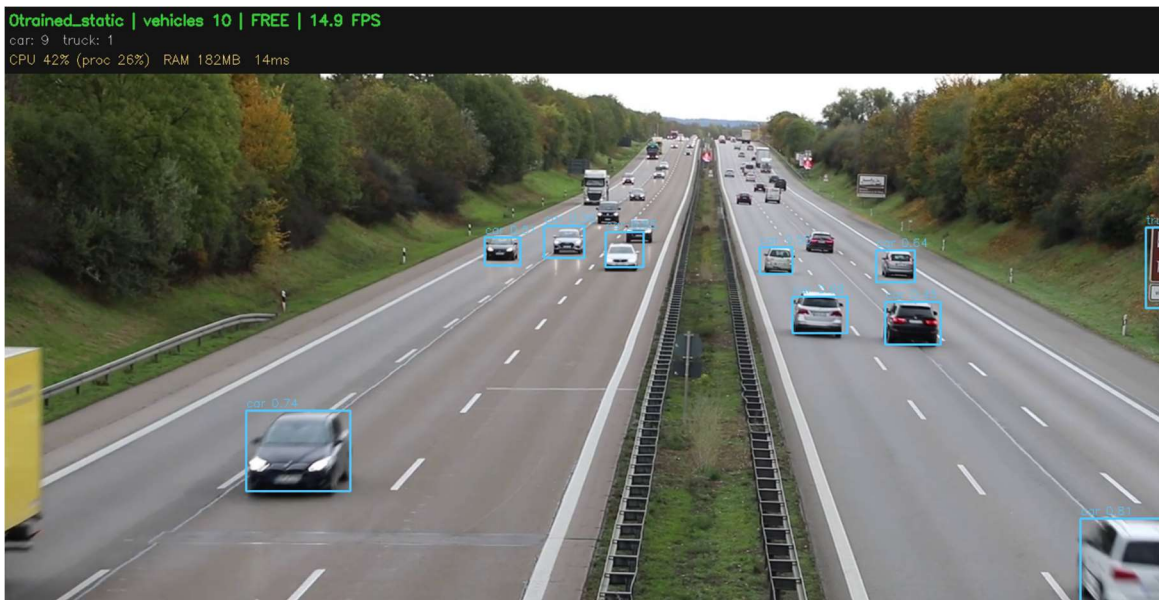
Contents	2
1 Abstract.....	3
2 Problem Statement	3
3 Dataset Details.....	4
4 Dataset Pre-processing.....	4
5 Baseline Model Architecture & Performance	4
6 Baseline Model Training / Inference Details	5
7 Hardware Details of Baseline Model Inference.....	6
8 Optimization Techniques Used.....	6
9 Comparative Results.....	8
10 Target Device Details	13
11 Deployment Challenges	13
12 Conclusion	14
13 Repository Link	14
14 References.....	14

(In Word: right-click the list → Update Field → Update entire table to fill page numbers.)

1 Abstract

This project builds an end-to-end embedded-AI system that detects vehicles and estimates road-traffic congestion in real time on a Raspberry Pi 5, intended for a Safe City deployment in which junction cameras drive adaptive traffic-signal timing.

A COCO-pretrained YOLOv8n detector is taken as the baseline (fine-tuned 20 epochs) and compressed for the CPU-only edge using two techniques: structured channel pruning (10%, 20%, 30%) and INT8 / FP16 quantization. Twelve ONNX variants are benchmarked on both a development laptop and the Pi 5 itself, measuring accuracy, latency, model size and resource usage at a fixed 15 FPS. The baseline INT8 model (0trained_static) is the best deployable result — full accuracy (~8.3 vehicles/frame), 6.16 MB, and only 78% CPU to hold 15 FPS on the Pi versus 92% for FP32.



Live detection on the Raspberry Pi 5 — INT8 model, 10 vehicles, congestion FREE, ~15 FPS, with the on-frame CPU / RAM / latency overlay.

2 Problem Statement

Urban intersections in a Safe City programme are already fitted with cameras, but fixed-timing traffic signals cannot respond to changing load. The goal is to measure congestion per junction in real time — vehicle counts and a FREE / MODERATE / HEAVY state — and feed it to the signal controllers so green / red timings adapt automatically.

Streaming every camera's video to a central server is bandwidth-heavy and does not scale across a city. The alternative is inference at the edge: run the detector on a cheap device next to each camera. A Raspberry Pi 5 is the target, but a full-precision YOLOv8n is too heavy for its CPU to run at a useful frame rate. The core problem is therefore to compress the detector so it runs in real time on the Pi while preserving detection accuracy, and to quantify the size / speed / accuracy trade-offs of each optimization.

3 Dataset Details

Two data sources are used — one for training / calibration and one for in-context inference verification.

Dataset	Role	Notes
vehicles_coco	Baseline training, pruning recovery, INT8 calibration	548 images (469 train / 79 val) in COCO format, keeping COCO's class scheme so vehicles stay at car=2, motorcycle=3, bus=5, truck=7 — a larger, traffic-focused sample than the 128-image COCO128 default.
Demo highway clips	Accuracy / latency verification	Oblique road-camera traffic videos (highway-busy.mp4) matching the deployment viewpoint; accuracy = average vehicles/frame on the busy clip.

Dataset lesson — match viewpoint & class scheme A raw aerial drone set (VisDrone) transferred poorly to the oblique road-camera view, so the training sample (vehicles_coco) keeps an oblique road perspective and COCO's class indices — a 0-indexed custom set would make the COCO-pretrained model detect nothing until the IDs were remapped.

4 Dataset Pre-processing

Every frame passes through the same deterministic pipeline before inference:

- **Letterbox resize** to 320×320 — aspect ratio preserved, remaining area grey-padded (no distortion of vehicle shapes).
- **Colour + layout** — BGR→RGB, pixel values normalised to [0,1], reordered to NCHW.
- **Class filtering** — detections kept only for COCO ids 2 / 3 / 5 / 7 (vehicles).
- **INT8 calibration set** — 150 representative images from the vehicle dataset are run through the model so the static quantizer learns per-tensor activation ranges.
- **Pruning recovery data** — the same training set is reused for the post-pruning fine-tune.

5 Baseline Model Architecture & Performance

YOLOv8n (Ultralytics) is a single-stage, anchor-free detector: a CSP-Darknet backbone built from C2f blocks, a PAN-FPN neck for multi-scale fusion, and a decoupled detection head (model.22) that predicts boxes and class scores at three strides. It is the smallest YOLOv8 variant (~3.2 M parameters, 8.7 GFLOPs at 640 px), the natural starting point for an edge deployment.

Reproduced performance vs. reported

Metric	Reported (Ultralytics, COCO val, 640 px)	Reproduced (this project)
Parameters	3.2 M	3.2 M
COCO mAP@50-95	37.3	see note
Input size	640×640	320×320 (edge)
FP32 model size (ONNX)	~12 MB	12.72 MB
Accuracy on busy-highway clip	—	~8.3 vehicles / frame
CPU latency (per frame)	~80 ms (ONNX-CPU ref.)	13.6 ms laptop · 36.6 ms Pi 5

Why we report a vehicles/frame proxy The published 37.3 mAP is measured on full COCO at 640 px. Our deployment runs at 320 px on road video for which there are no ground-truth labels, so the in-context accuracy metric is the average number of vehicles detected per frame on a fixed busy-highway clip — the quantity the Safe City controller actually consumes, used consistently to compare every optimized variant.

6 Baseline Model Training / Inference Details

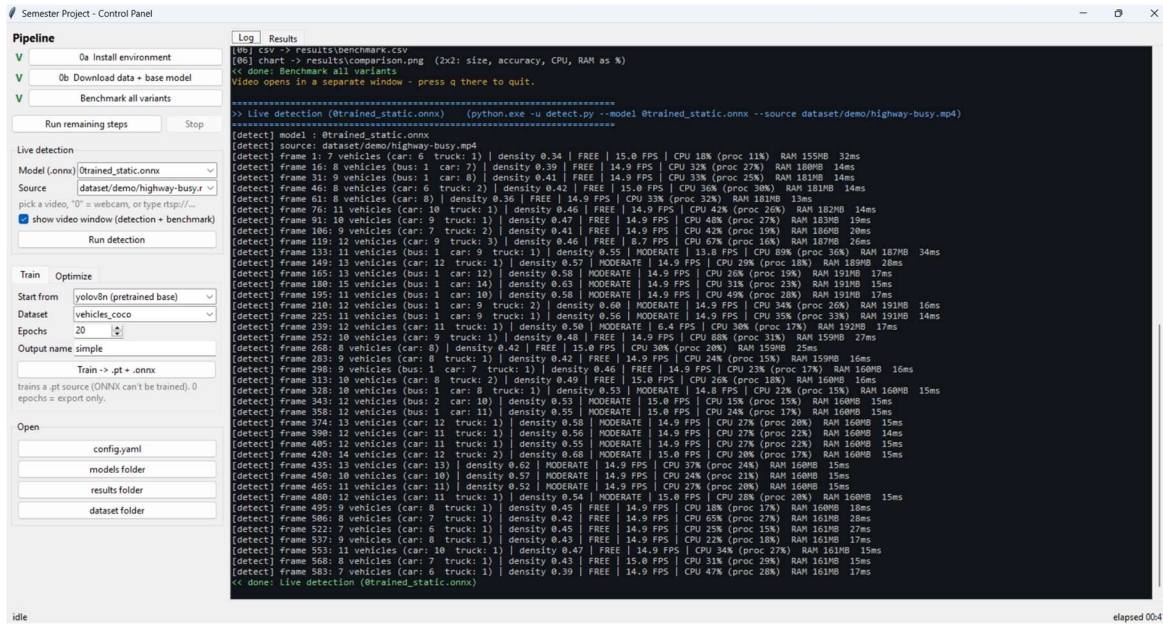
Training

- Start from `yo1ov8n.pt` (COCO-pretrained weights).
- Fine-tune **20 epochs** on the 548-image `vehicles_coco` set at `imgsz 320`; the result is named `θtrained` (0% pruned).
- Export to **ONNX, opset 13** (required for the per-channel INT8 QDQ used later).

Inference

- **Runtime:** ONNX Runtime, CPU execution provider (no GPU on the Pi).
- **Post-processing:** confidence threshold + non-max suppression, then vehicle-class filtering.
- **Congestion:** in-ROI vehicle density is smoothed and thresholded into FREE / MODERATE / HEAVY.
- **Instrumentation:** a live overlay reports vehicles, FPS and CPU / RAM / inference-ms; the rate is capped at a fixed 15 FPS so every model is compared under identical load.

The whole pipeline — train, prune, quantize, benchmark and live-detect — is driven from a single control-panel GUI (`gui.py`), with a headless launcher (`run_on_pi.sh`) for the Pi.



The Control Panel (gui.py): pipeline + live-detection picker + Train / Optimize tabs on the left; the live Log on the right.

7 Hardware Details of Baseline Model Inference

A PC does the heavy lifting (training, pruning, quantization, and the laptop benchmark); the Raspberry Pi 5 runs the deployed inference.

Role	Device	Details
Development / optimization	Laptop	Intel Core i7-1165G7 (Tiger Lake, 4 cores / 8 threads), CPU-only inference, Windows 11.
Edge deployment (target)	Raspberry Pi 5	Broadcom BCM2712 — quad-core Arm Cortex-A76 @ 2.4 GHz, LPDDR4X RAM, Raspberry Pi OS (64-bit, Bookworm), ONNX Runtime CPU. No GPU / NPU.

Both run inference purely on the CPU — the headline reason the study benchmarks on the Pi and not only on the laptop: INT8 acceleration behaves very differently on x86 vs. the Pi's Arm NEON cores.

8 Optimization Techniques Used

8.1 Quantization

Quantization lowers the numeric precision of the weights (and, for INT8, the activations).

- **FP16 (half precision)** — near-lossless, halves the file (12.72→6.4 MB); no CPU speed-up but smaller and useful where memory is tight.
- **INT8 static (selective)** — calibrated 8-bit weights/activations, per-channel weight scales (QDQ, opset 13). The detection head (66 nodes under /model.22/) is kept FP32; quantizing it collapses detections to zero. This selective INT8 is the deployable quantized model.

Why full INT8 breaks YOLO The detection head emits wide-range box/score values that 8-bit cannot represent; every prediction falls below threshold and detection drops to zero. Excluding the 66 head nodes recovers the detections at essentially the same size.

8.2 Pruning

Structured channel pruning (torch-pruning, grouped-L2 importance) removes the least-important convolution channels — and every weight that depends on them — at 10%, 20% and 30%. The detection head is left intact, then a short recovery fine-tune follows.

Critical finding — gentle recovery learning rate A pruned model must be fine-tuned to recover, but at a low learning rate (0.0005). The default training rate (0.01) re-trains the network and erases the pretrained features — this single change took the pruned model from 0 to ~6.5 vehicles/frame.

Correct order: prune first (still trainable, can recover), then quantize (freezes the graph). So 30pruned_static.onnx is 30%-pruned then INT8-quantized.

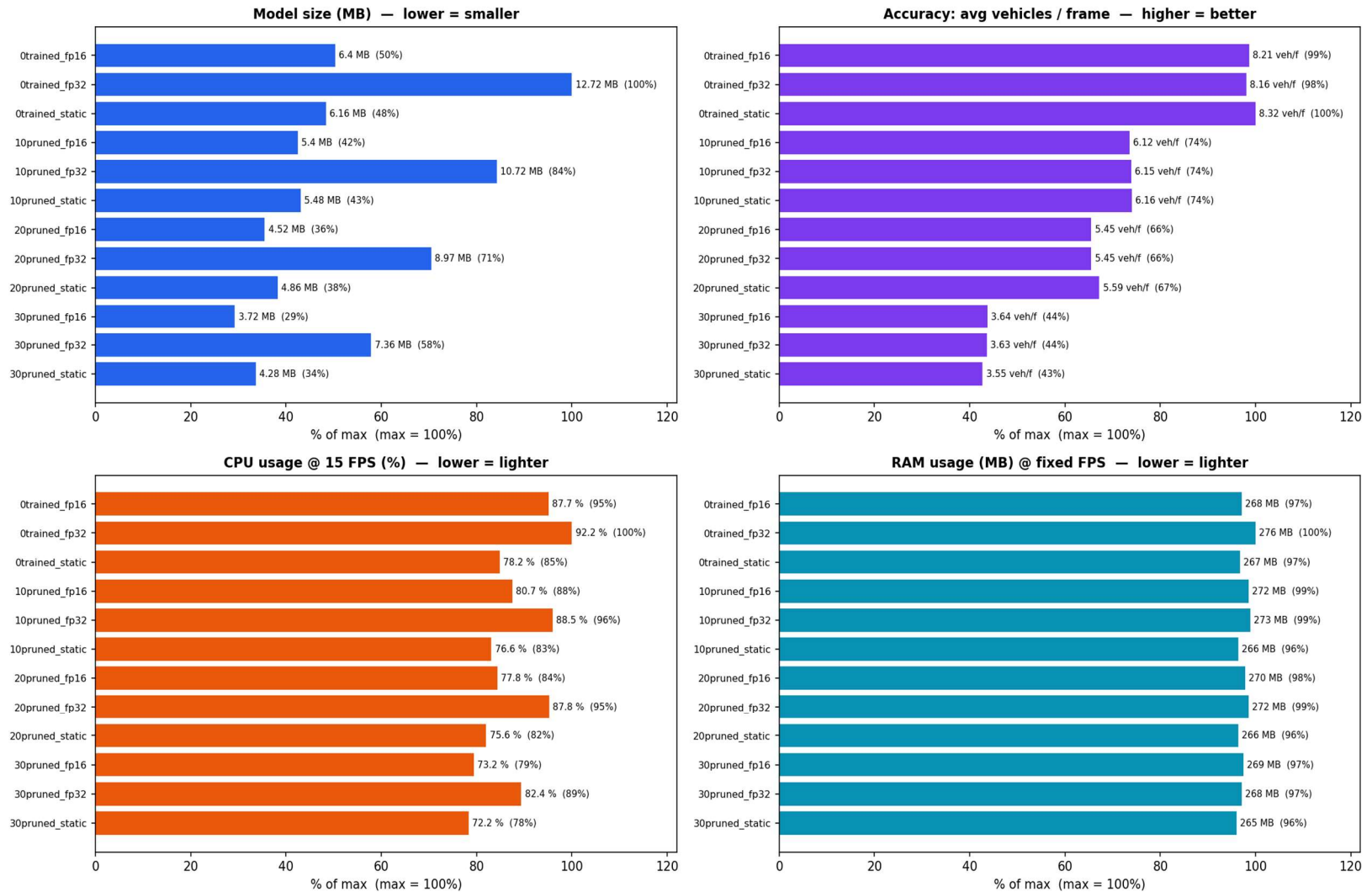
Hardware-aware NAS — investigated, not deployed A depth/width architecture search timed on ONNX Runtime was considered as a third technique, but its per-candidate retraining is GPU-intensive and impractical on the CPU-only workflow used here, so the deployed comparison covers the baseline plus pruning and quantization.

9 Comparative Results

All 12 ONNX variants (baseline 0trained and 10 / 20 / 30 pruned, each at _fp32 / _fp16 / _static) were benchmarked at a fixed 15 FPS on the busy-highway clip — first on the laptop, then on the Pi 5. Each bar in the charts is a percentage of the maximum in its metric (max = 100%).

9.1 On the Raspberry Pi 5 (real on-device numbers)

Model comparison — all variants @ fixed 15 FPS (bars = % of max)



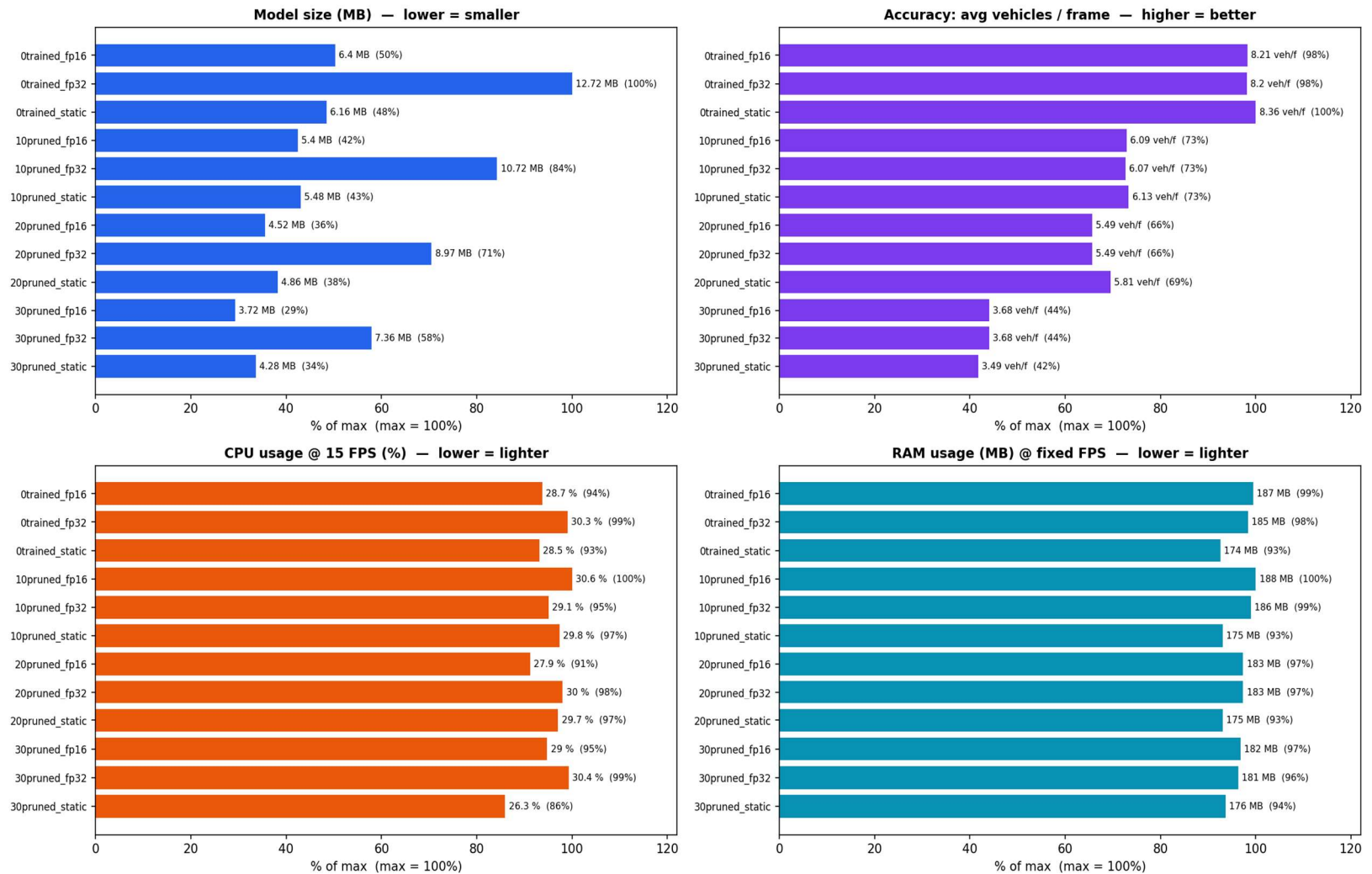
Pi 5 — model size, accuracy, CPU and RAM per variant (each bar as % of max).

Model	Size (MB)	Pi infer (ms)	Acc (veh/f)	CPU% @15	RAM (MB)
0trained_fp16.onnx	6.4	30.1	8.21	87.7	268.0
0trained_fp32.onnx	12.72	36.6	8.16	92.2	276.0
0trained_static.onnx	6.16	24.6	8.32	78.2	267.0
10pruned_fp16.onnx	5.4	25.7	6.12	80.7	272.0
10pruned_fp32.onnx	10.72	61.0	6.15	88.5	273.0
10pruned_static.onnx	5.48	23.3	6.16	76.6	266.0
20pruned_fp16.onnx	4.52	22.9	5.45	77.8	270.0
20pruned_fp32.onnx	8.97	29.0	5.45	87.8	272.0
20pruned_static.onnx	4.86	23.3	5.59	75.6	266.0
30pruned_fp16.onnx	3.72	22.5	3.64	73.2	269.0
30pruned_fp32.onnx	7.36	26.4	3.63	82.4	268.0
30pruned_static.onnx	4.28	20.5	3.55	72.2	265.0

Verdict On the Pi the baseline INT8 (0trained_static) wins: ~25 ms/inference (~40 FPS of raw headroom), only 78% CPU to hold 15 FPS vs 92% for FP32, 6.16 MB, full 8.3 vehicles/frame, and the lowest RAM (~267 MB). INT8 runs ~1.5x faster than FP32 because Arm's NEON path executes 8-bit dot-products natively.

9.2 On the development laptop

Model comparison — all variants @ fixed 15 FPS (bars = % of max)



Laptop — same four metrics. INT8 shows little speed-up on x86 (no VNNI), unlike on the Pi.

Model	Size (MB)	Latency (ms)	Acc (veh/f)	CPU% @15	RAM (MB)
0trained_fp16.onnx	6.4	12.3	8.21	28.7	187.0
0trained_fp32.onnx	12.72	13.6	8.2	30.3	185.0
0trained_static.onnx	6.16	11.7	8.36	28.5	174.0
10pruned_fp16.onnx	5.4	14.2	6.09	30.6	188.0
10pruned_fp32.onnx	10.72	15.7	6.07	29.1	186.0
10pruned_static.onnx	5.48	12.9	6.13	29.8	175.0
20pruned_fp16.onnx	4.52	13.0	5.49	27.9	183.0
20pruned_fp32.onnx	8.97	14.2	5.49	30.0	183.0
20pruned_static.onnx	4.86	12.7	5.81	29.7	175.0
30pruned_fp16.onnx	3.72	10.5	3.68	29.0	182.0
30pruned_fp32.onnx	7.36	11.9	3.68	30.4	181.0
30pruned_static.onnx	4.28	10.3	3.49	26.3	176.0

9.3 Reading the four dimensions

Dimension	Result
Accuracy	Quantization preserves it (~8.3 veh/f); pruning trades it steeply (10%→~6.1, 20%→~5.5, 30%→~3.6).
Latency (Pi)	FP32 36.6 ms → FP16 30.1 ms → INT8 24.6 ms (≈1.5x faster).
Model size	FP32 12.72 MB → INT8 6.16 MB (2x) → 30%-pruned INT8 4.28 MB.
Resource usage (Pi)	INT8 needs the least CPU (78% vs 92%) and RAM (~267 MB) to sustain 15 FPS.

10 Target Device Details

The deployment target is the Raspberry Pi 5 — an ~\$60-80 single-board computer that is a realistic per-junction edge node for a Safe City rollout.

Component	Specification
SoC	Broadcom BCM2712
CPU	Quad-core Arm Cortex-A76 @ 2.4 GHz (64-bit)
Memory	LPDDR4X-4267 (4 GB / 8 GB)
OS	Raspberry Pi OS (64-bit, Debian Bookworm)
Inference runtime	ONNX Runtime, CPU execution provider
Input	Local demo video (no camera / LAN / ONVIF required); RTSP optional

Deployment is a folder copy: install the lightweight inference dependencies (`python3 install.py --edge`) and run `bash run_on_pi.sh`, which auto-detects whether a display is present and otherwise runs headless.

11 Deployment Challenges

Getting the optimized models to run cleanly on the Pi surfaced several real, non-obvious problems:

Challenge	Cause & resolution
No video window on the Pi	pip's Arm opencv-python is a headless build (no <code>cv2.imshow</code>). Fixed by installing Debian's <code>python3-opencv</code> and exposing it to the venv with <code>--system-site-packages</code> .
Benchmark chart never generated	matplotlib was missing from the edge install; added to the Pi dependency set.
Chart wouldn't open	The Pi's default image viewer (Eye of MATE) was broken; the GUI now renders the chart in its own in-app window.
Full INT8 detected nothing	The detection head must stay FP32 (selective INT8) — quantizing it zeroes detections.
FP16 export produced an invalid graph	onnxconverter-common mishandled YOLO Resize/Concat; switched to the ONNX-Runtime transformers float16 converter.
INT8 graph rejected (axis for DequantizeLinear)	Per-channel INT8 needs opset 13; the export opset was raised from 12 to 13.
Pruned model detected nothing	Recovery fine-tune at the default LR destroyed the pretrained features; a gentle LR (0.0005) fixed it.
No GPU on the edge workflow	Hardware-aware NAS (which retrains each candidate) was not feasible and was left as future work.

12 Conclusion

The project delivers a complete, working embedded-AI pipeline: a COCO-pretrained YOLOv8n detector compressed for the Raspberry Pi 5 through structured pruning and INT8 / FP16 quantization, benchmarked on real hardware, and wrapped in a single GUI that trains, optimizes, benchmarks and runs live detection with a resource overlay.

Across 12 variants measured on the Pi, the clear recommendation is the baseline INT8 model (0trained_static): it keeps full accuracy (~8.3 veh/frame), halves the model to 6.16 MB, and needs the least CPU and RAM to hold 15 FPS — comfortably real-time on a ~\$70 device. Pruning shrinks the model further but costs accuracy steeply, so it is reserved for cases where every megabyte counts. The work also surfaced reusable lessons: the gentle learning rate for pruning recovery, the detection head that must stay full-precision under INT8, and the necessity of benchmarking on the target Arm hardware.

For the Safe City application, each junction can run vehicle detection and congestion estimation locally on a Pi and report just the per-lane counts and congestion state to the traffic-signal controllers — low bandwidth, scalable, real-time. Future work: hardware-aware NAS on a GPU, a larger real traffic-camera dataset, and live integration with a signal-controller test-bed.

Prepared by Engr Danial Azam (PHDEE25003) for Prof Rehan Hafiz · Embedded AI.

13 Repository Link

The full code — baseline reproduction, optimization, and deployment — is in the project GitHub repository.

Repository: <https://github.com/DanialAzam/EmbeddedAI>

Clone: `git clone https://github.com/DanialAzam/EmbeddedAI.git`

Purpose	Code
Reproduce the baseline	download.py, 01_simple_model.py
Optimization	02_pruned_model.py, 03_quantized_model.py, quantize_levels.py
Benchmark	06_benchmark.py
Deployment / inference	detect.py, run_on_pi.sh, install.py
Control-panel GUI	gui.py

14 References

1. G. Jocher, A. Chaurasia, J. Qiu. Ultralytics YOLOv8, 2023. github.com/ultralytics/ultralytics
2. G. Fang et al. “DepGraph: Towards Any Structural Pruning” (Torch-Pruning). CVPR, 2023.
3. B. Jacob et al. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference.” CVPR, 2018.
4. Microsoft. ONNX Runtime. onnxruntime.ai
5. T.-Y. Lin et al. “Microsoft COCO: Common Objects in Context.” ECCV, 2014.
6. Open Neural Network Exchange (ONNX). onnx.ai
7. Raspberry Pi Ltd. Raspberry Pi 5 product documentation. raspberrypi.com/documentation